PYTHON DSA BOOTCAMP

Day 3 Training Manual

Topic: TREES

Tree Fundamentals | Binary Tree | BST | Tree Traversals | Coding Practice

Total Duration: 6 hrs 20 min (2 sessions + practical)

Day 3 Schedule

Time	Duration	Block	Focus
9:10 – 10:50	1h 40m	Session 1	Tree Fundamentals: definition, terminology, properties, types, memory representation
10:50 – 11:00	10 min	Tea Break	—
11:00 – 12:40	1h 40m	Session 2	Binary Tree, BST: insertion, deletion, search + all 5 Tree Traversals (DFS & BFS)
12:40 – 1:30	50 min	Lunch Break	—
1:30 – 4:00	2h 30m	Practical Session	BST implementation from scratch + 3 LeetCode problems: Max Depth, Same Tree, Invert Binary Tree

Table of Contents

Table of Contents 2
Part 1: Tree Fundamentals (Session 1 — 1h 40m) 4
1.1 What is a Tree? 4
1.2 Tree Terminologies — Complete Glossary 4
1.3 Reference Tree for Terminologies 5
1.4 Types of Trees 6
1.5 Tree Properties — Key Formulas 6
1.6 Memory Representation of Binary Trees 6
(a) Linked Representation — Node Class (Most Common) 6
(b) Array Representation — Level Order Indexing 7
Part 2: Binary Tree, BST and Traversals (Session 2 — 1h 40m) 8
2.1 Binary Tree — Formal Definition and Properties 8
2.2 Binary Search Tree (BST) 8
BST Operations Summary 8
Complete BST Implementation in Python 9
2.3 Tree Traversals — All 5 Methods 10
DFS Traversal 1 — Inorder (Left → Root → Right) 11
DFS Traversal 2 — Preorder (Root → Left → Right) 11
DFS Traversal 3 — Postorder (Left → Right → Root) 12
BFS Traversal 4 — Level Order (Left to Right, Level by Level) 12
BFS Traversal 5 — Reverse Level Order (Bottom-Up) 13
2.4 Traversal Comparison Table 13
Part 3: Practical Session — Coding (1:30 – 4:00) 15
3.1 10 Interview Questions (Warm-up — 30 minutes) 15
Q1. What is the difference between a Tree and a Graph? 15
Q2. What is the difference between height and depth of a node? 15
Q3. What are the three DFS traversal orders? When would you use each? 15
Q4. What data structure does BFS use? What does DFS use? Why? 15
Q5. How do you check if a binary tree is a valid BST? 16
Q6. What is the time and space complexity of the three DFS traversals? 16
Q7. What are the three cases for deleting a node in a BST? 16
Q8. What is the difference between a complete binary tree, a full binary tree, and a perfect binary tree? 16
Q9. In an inorder traversal of a BST, can you find the kth smallest element? What is the complexity? 16
Q10. What is the relationship between tree height and worst-case BST performance? 16
3.2 Coding Problems 17

Problem 1 — Maximum Depth of Binary Tree (LeetCode 104) 17
Problem Statement 17
Approach 1 — DFS Recursion (Postorder) 17
Approach 2 — BFS (Count Levels) 17
Dry Run — DFS on [3, 9, 20, null, null, 15, 7] 18
Complexity Analysis 18
Follow-up Questions 18
Problem 2 — Same Tree (LeetCode 100) 20
Problem Statement 20
Approach — Simultaneous DFS 20
Approach 2 — BFS with Twin Queue 21
Dry Run — p=[1,2,3], q=[1,2,3] 21
Dry Run — p=[1,2], q=[1,null,2] (should return False) 21
Complexity Analysis 21
Problem 3 — Invert Binary Tree (LeetCode 226) 23
Problem Statement 23
Approach 1 — DFS Recursive (Postorder) 23
Approach 2 — BFS (Level-by-Level Swap) 24
Dry Run — DFS on [4, 2, 7, 1, 3, 6, 9] 24
Complexity Analysis 24
Common Mistakes to Watch For 24
Part 4: Complete BST Implementation Task (Practical — 45 min) 26
Part 5: Recap, Bonus Problems & Homework 30
5.1 Day 3 Key Takeaways 30
5.2 Traversal Quick Reference Card 30
5.3 Bonus / Extra Practice Problems 30
5.4 Homework Assignment 30

9:10 – 10:50 Session 1	Tree Fundamentals & Tree Terminologies

Part 1: Tree Fundamentals (Session 1 — 1h 40m)

1.1 What is a Tree?

A Tree is a non-linear, hierarchical data structure consisting of nodes connected by edges. Unlike Arrays, Linked Lists, Stacks and Queues which are linear (one after the other), a Tree branches out — one node can connect to multiple children, forming a parent-child hierarchy.

- **Formal definition:** A tree with n nodes has exactly n-1 edges. There is exactly one path between any two nodes, and there is no cycle.
- **Real-life analogies:** File system (folders and subfolders), company org chart, HTML DOM, family tree, tournament bracket, decision tree in ML.

> **Tip: Teaching Tip**
> Draw a company org chart on the board first — CEO at top, departments below, teams under departments. This makes every tree terminology feel natural and intuitive before you introduce any code.

1.2 Tree Terminologies — Complete Glossary

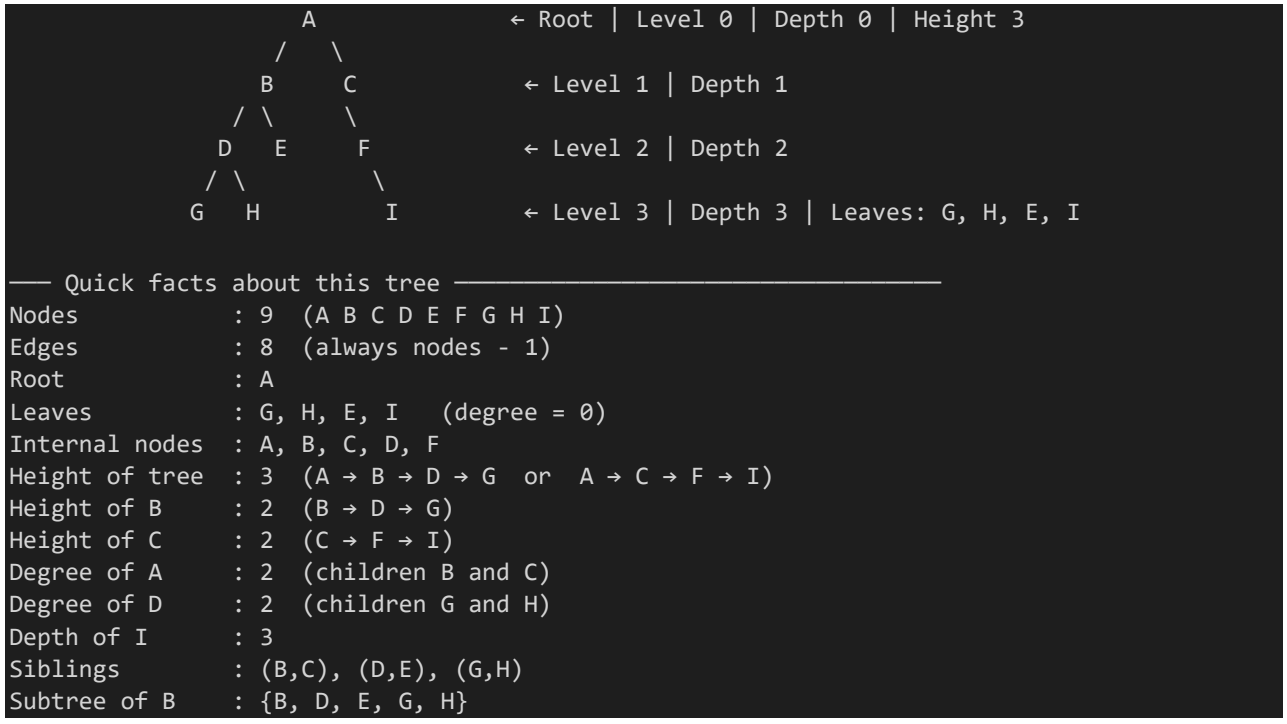
Every term below should be demonstrated on the same diagram drawn on the board:

Term	Definition	Example / Analogy
Node	Basic unit of a tree; contains data and references to children	Each person in an org chart is a node
Root	The topmost node; the only node with no parent	CEO in an org chart
Edge	A connection (link) between a parent and a child node	Reporting line between manager and employee
Parent	A node that has one or more children	A manager who has direct reports
Child	A node that has a parent	A direct report of a manager
Leaf / External Node	A node with NO children (degree = 0)	An individual contributor with no reports
Internal Node	A node that has at least one child	Any manager in the hierarchy
Sibling	Nodes that share the same parent	Two employees reporting to the same manager
Subtree	A node and all its descendants form a subtree	A department and all its sub-teams

Term	Definition	Example / Analogy
Depth of a Node	Number of edges from the root to that node (root depth = 0)	CEO is depth 0, VP is depth 1
Height of a Node	Number of edges on the longest path from that node to a leaf	Height of a leaf = 0
Height of Tree	Height of the root node (= max depth of any node)	Levels from CEO to lowest employee
Level	All nodes at the same depth form a level (Level 0 = root)	All VPs form Level 1
Degree	Number of children a node has	A node with 3 children has degree 3
Degree of Tree	Maximum degree among all nodes in the tree	Binary tree has max degree 2
Ancestor	All nodes on the path from root to a given node	Root, and all managers above you
Descendant	All nodes reachable downward from a given node	All direct and indirect reports
Path	Sequence of nodes and edges connecting two nodes	Reporting chain from CEO to an employee
Forest	A collection of disjoint trees	Multiple separate org charts

1.3 Reference Tree for Terminologies

Use this tree structure throughout Session 1 when demonstrating every term above:



```
Ancestors of G  : A, B, D
```

1.4 Types of Trees

Tree Type	Key Property	Common Use
General Tree	Each node can have any number of children	File systems, org charts
Binary Tree	Each node has at most 2 children (left, right)	Expression trees, BST, heaps
Binary Search Tree	Left child < parent < right child	Searching, sorted data
Full Binary Tree	Every node has 0 or exactly 2 children	Expression trees
Complete Binary Tree	All levels filled except possibly last; last level filled left to right	Binary Heap
Perfect Binary Tree	All internal nodes have 2 children; all leaves at same level	Theoretical reference
Balanced Binary Tree	Height difference of left and right subtree ≤ 1 for every node	AVL Tree, Red-Black Tree
AVL Tree	Self-balancing BST; height always O(log n)	Databases, maps
N-ary Tree	Each node has at most N children	Tries, B-trees
Trie (Prefix Tree)	Each edge represents a character	Autocomplete, spell check
Segment Tree	Each node stores aggregate of a range	Range query problems
Heap	Complete binary tree with min/max property	Priority queues

1.5 Tree Properties — Key Formulas

Property	Formula	Notes
Max nodes at level L	2^L	Root is level 0; a perfect BT at level 3 has 8 nodes
Max nodes in BT of height H	2^(H+1) - 1	All levels fully filled
Min height for n nodes (BT)	$\lfloor \log_2(n) \rfloor$	Best case: perfectly balanced
Max height for n nodes (BT)	n - 1	Worst case: skewed (like a linked list)
Number of leaf nodes in full BT	⌈n/2⌉	n = total nodes
Edges in any tree	n - 1	Always true for any tree with n nodes

1.6 Memory Representation of Binary Trees

(a) Linked Representation — Node Class (Most Common)

Each node is an object with a data value and two pointers — one to the left child and one to the right child. Null/None pointers represent absent children.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val   = val
        self.left  = left      # pointer to left child (or None)
        self.right = right     # pointer to right child (or None)

# Manually building the tree:
#        1
#       / \
#      2   3
#     / \
#    4   5

root      = TreeNode(1)
root.left = TreeNode(2)
root.right= TreeNode(3)
root.left.left  = TreeNode(4)
root.left.right = TreeNode(5)
```

(b) Array Representation — Level Order Indexing

A binary tree can be stored in an array where the root is at index 0 (or 1 for 1-indexed), and for any node at index i:

- **Left child index:** 2*i + 1 (0-indexed) or 2*i (1-indexed)
- **Right child index:** 2*i + 2 (0-indexed) or 2*i+1 (1-indexed)
- **Parent index:** (i-1)//2 (0-indexed) or i//2 (1-indexed)

```
# Tree:       1
#            / \
#           2   3
#          / \
#         4   5
#
# Array representation (0-indexed):
# Index:  0  1  2  3  4
# Array: [1, 2, 3, 4, 5]
#
# Verification:
# Node at i=1 (val=2): left child = 2*1+1 = 3 (val=4) ✓
#                      right child= 2*1+2 = 4 (val=5) ✓
# Node at i=3 (val=4): parent = (3-1)//2 = 1 (val=2) ✓
```

Note: When to use array rep

Array representation works best for complete binary trees (like heaps) where there are no gaps. For sparse trees, many indices will be None/null, wasting memory. Linked representation is preferred for general BSTs and tree problems.

11:00 – 12:40 Session 2	Binary Tree, BST and Tree Traversals

Part 2: Binary Tree, BST and Traversals (Session 2 — 1h 40m)

2.1 Binary Tree — Formal Definition and Properties

A Binary Tree is a tree where every node has at most two children, conventionally referred to as the left child and the right child. There is no constraint on the ordering of values — that is what distinguishes it from a BST.

Type	Property	Height	Search
Skewed Binary Tree	All nodes have only left (or only right) child — degenerates to a linked list	O(n)	O(n)
Full Binary Tree	Every node has exactly 0 or 2 children	O(log n) to O(n)	O(n)
Complete Binary Tree	All levels filled except possibly last; last level filled left to right	O(log n)	O(n)
Perfect Binary Tree	All internal nodes have 2 children; all leaves at same depth	O(log n)	O(n)
Balanced Binary Tree	|height(left) − height(right)| ≤ 1 for every node	O(log n)	O(log n) in BST

2.2 Binary Search Tree (BST)

A Binary Search Tree is a binary tree with a strict ordering property: for any node N, all values in its left subtree are less than N's value, and all values in its right subtree are greater than N's value. This holds recursively for every subtree.

Key Insight: The BST Invariant

Left subtree values < Node value < Right subtree values — this must hold for EVERY node, not just the immediate children. A common mistake is checking only one level deep. The classic interview trap: a node might be out of place two or three levels down.

BST Operations Summary

Operation	Average Case	Worst Case (Skewed)	Notes
Search	O(log n)	O(n)	Binary search: go left if smaller, right if larger
Insert	O(log n)	O(n)	Insert at correct leaf position; never restructures
Delete	O(log n)	O(n)	Three cases: leaf, one child, two children (inorder successor)

Operation	Average Case	Worst Case (Skewed)	Notes
Min value	O(log n)	O(n)	Leftmost node in the tree
Max value	O(log n)	O(n)	Rightmost node in the tree
Inorder traversal	O(n)	O(n)	Produces sorted sequence — unique to BSTs
Space	O(n)	O(n)	For n nodes

Complete BST Implementation in Python

```
class TreeNode:
    def __init__(self, val):
        self.val   = val
        self.left  = None
        self.right = None


class BST:
    def __init__(self):
        self.root = None

    # ── INSERT ──────────────────────────────────────────────
    def insert(self, val):
        self.root = self._insert(self.root, val)

    def _insert(self, node, val):
        if node is None:                    # found the correct empty slot
            return TreeNode(val)
        if val < node.val:
            node.left  = self._insert(node.left,  val)
        elif val > node.val:
            node.right = self._insert(node.right, val)
        # if val == node.val: ignore duplicates (BST invariant: unique values)
        return node

    # ── SEARCH ──────────────────────────────────────────────
    def search(self, val):
        return self._search(self.root, val)

    def _search(self, node, val):
        if node is None:       return False   # not found
        if node.val == val:    return True    # found
        if val < node.val:     return self._search(node.left,  val)
        return                        self._search(node.right, val)

    # ── DELETE ──────────────────────────────────────────────
    def delete(self, val):
        self.root = self._delete(self.root, val)

    def _delete(self, node, val):
        if node is None:
            return node                       # val not in tree
        if val < node.val:
```

```
            node.left  = self._delete(node.left,  val)
        elif val > node.val:
            node.right = self._delete(node.right, val)
        else:
            # Case 1: leaf node
            if not node.left and not node.right:
                return None
            # Case 2: one child
            if not node.left:   return node.right
            if not node.right:  return node.left
            # Case 3: two children → replace with inorder successor
            successor = self._min_node(node.right)
            node.val  = successor.val
            node.right= self._delete(node.right, successor.val)
        return node

    def _min_node(self, node):
        while node.left:
            node = node.left
        return node

    # ── MIN / MAX ──────────────────────────────────────────
    def minimum(self):
        if not self.root: return None
        node = self.root
        while node.left: node = node.left
        return node.val

    def maximum(self):
        if not self.root: return None
        node = self.root
        while node.right: node = node.right
        return node.val


# ── QUICK TEST ─────────────────────────────────────────────
bst = BST()
for v in [5, 3, 7, 1, 4, 6, 8]:
    bst.insert(v)

#       5
#      / \
#     3   7
#    / \ / \
#   1  4 6  8

print(bst.search(4))   # True
print(bst.search(9))   # False
print(bst.minimum())   # 1
print(bst.maximum())   # 8
```

2.3 Tree Traversals — All 5 Methods

A traversal is a systematic way to visit every node in a tree exactly once. The order in which nodes are visited defines the traversal type. There are two families: DFS (Depth-First Search — goes deep before wide) and BFS (Breadth-First Search — visits level by level).

Reference tree for all traversal examples:

```
             4
            / \
           2   6
          / \ / \
         1  3 5  7

Expected outputs:
  Inorder    : 1 2 3 4 5 6 7   ← sorted order (unique to BST)
  Preorder   : 4 2 1 3 6 5 7   ← root first
  Postorder  : 1 3 2 5 7 6 4   ← root last
  Level Order: 4 2 6 1 3 5 7   ← row by row
  Reverse LO : 1 3 5 7 2 6 4   ← bottom-up level order
```

DFS Traversal 1 — Inorder (Left → Root → Right)

Visits: left subtree first, then the root, then the right subtree. Produces nodes in sorted ascending order for a BST — this is a key property used in many interview problems.

```
# Recursive
def inorder(node):
    if node is None:
        return
    inorder(node.left)
    print(node.val, end=' ')
    inorder(node.right)

# Iterative (using explicit stack)
def inorder_iterative(root):
    result, stack = [], []
    current = root
    while current or stack:
        while current:               # go as left as possible
            stack.append(current)
            current = current.left
        current = stack.pop()        # backtrack
        result.append(current.val)
        current = current.right      # move to right subtree
    return result

# Output: [1, 2, 3, 4, 5, 6, 7]
```

DFS Traversal 2 — Preorder (Root → Left → Right)

Visits the root first, then recursively traverses left, then right. Used to copy/serialize a tree, create a prefix expression tree, or explore decisions before children.

```
# Recursive
def preorder(node):
    if node is None:
        return
```

```
    print(node.val, end=' ')
    preorder(node.left)
    preorder(node.right)

# Iterative
def preorder_iterative(root):
    if not root: return []
    result, stack = [], [root]
    while stack:
        node = stack.pop()
        result.append(node.val)
        if node.right: stack.append(node.right)  # right first (LIFO)
        if node.left:  stack.append(node.left)
    return result

# Output: [4, 2, 1, 3, 6, 5, 7]
```

DFS Traversal 3 — Postorder (Left → Right → Root)

Visits both children before the root. Used for deleting a tree (children before parents), evaluating postfix expression trees, or computing directory sizes (compute subfolder sizes before the parent folder).

```
# Recursive
def postorder(node):
    if node is None:
        return
    postorder(node.left)
    postorder(node.right)
    print(node.val, end=' ')

# Iterative (reverse of modified preorder trick)
def postorder_iterative(root):
    if not root: return []
    result, stack = [], [root]
    while stack:
        node = stack.pop()
        result.append(node.val)
        if node.left:  stack.append(node.left)
        if node.right: stack.append(node.right)
    return result[::-1]             # reverse gives postorder

# Output: [1, 3, 2, 5, 7, 6, 4]
```

BFS Traversal 4 — Level Order (Left to Right, Level by Level)

Uses a queue to explore all nodes at depth d before any node at depth d+1. Essential for shortest-path problems on trees, connecting nodes at the same level, and returning results grouped by level.

```
from collections import deque

def level_order(root):
    if not root: return []
    result = []
    q = deque([root])
    while q:
        level_size = len(q)      # number of nodes at current level
```

```
        level = []
        for _ in range(level_size):
            node = q.popleft()
            level.append(node.val)
            if node.left:  q.append(node.left)
            if node.right: q.append(node.right)
        result.append(level)
    return result

# Output: [[4], [2, 6], [1, 3, 5, 7]]   (grouped by level)
# Flat:   [4, 2, 6, 1, 3, 5, 7]
```

BFS Traversal 5 — Reverse Level Order (Bottom-Up)

Same as level order, but prepend each level to the result (or append to a deque and reverse at the end). Returns nodes from the deepest level upward.

```
from collections import deque

def reverse_level_order(root):
    if not root: return []
    result = []
    q = deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left:  q.append(node.left)
            if node.right: q.append(node.right)
        result.append(level)
    return result[::-1]          # reverse the levels list

# Output: [[1, 3, 5, 7], [2, 6], [4]]
```

2.4 Traversal Comparison Table

Traversal	Order	Stack/Queue	Time	Space	Key Use Case
Inorder	Left, Root, Right	Stack (implicit)	O(n)	O(h)	BST sorted output, validate BST
Preorder	Root, Left, Right	Stack (implicit)	O(n)	O(h)	Copy tree, serialize, prefix expression
Postorder	Left, Right, Root	Stack (implicit)	O(n)	O(h)	Delete tree, postfix expression, folder sizes
Level Order (BFS)	Level by level L→R	Queue	O(n)	O(w)	Shortest path, connect levels, zigzag

Traversal	Order	Stack/Queue	Time	Space	Key Use Case
Reverse Level	Bottom levels first	Queue + reverse	O(n)	O(w)	Bottom-up aggregation problems

h = height of tree (O(log n) balanced, O(n) skewed) | w = max width of tree (at most O(n/2) for the widest level)

Tree Pattern: Quick Traversal Recall Trick

Think of the ROOT's position relative to left and right: PRE-order = ROOT first. IN-order = ROOT in the middle. POST-order = ROOT last. BFS uses a Queue; DFS uses a Stack (explicit or recursion call stack).

1:30 – 4:00 Practical Session	**Tree Problems + BST Implementation**

Part 3: Practical Session — Coding (1:30 – 4:00)

Problem	LeetCode #	Difficulty	Pattern	Est. Time
Maximum Depth of Binary Tree	104	Easy	DFS (recursive) / BFS (level count)	25 min
Same Tree	100	Easy	DFS simultaneous traversal / BFS queue	25 min
Invert Binary Tree	226	Easy	DFS post-order / BFS level swap	25 min
BST Implementation	—	Medium	Complete BST from scratch	45 min

3.1 10 Interview Questions (Warm-up — 30 minutes)

Quick fire round before coding. Ask each, let students answer, then clarify.

Q1. What is the difference between a Tree and a Graph?

Discussion Points: A tree is a connected acyclic graph with exactly n-1 edges for n nodes. A graph can have cycles and disconnected components. Every tree is a graph but not every graph is a tree. Trees have a designated root; general graphs do not.

Q2. What is the difference between height and depth of a node?

Discussion Points: Depth = edges from root DOWN to the node (root depth = 0). Height = edges from the node UP to its deepest leaf (leaf height = 0). Height of the tree = height of the root = maximum depth. A common confusion: check whether the interview question is 0-indexed or 1-indexed (LeetCode's max depth counts nodes, not edges).

Q3. What are the three DFS traversal orders? When would you use each?

Discussion Points: Inorder (L-Root-R): gives sorted output for BST, validate BST. Preorder (Root-L-R): serialize/copy tree, prefix expressions. Postorder (L-R-Root): delete tree safely, postfix expressions, compute dependent values (folder size) before the parent.

Q4. What data structure does BFS use? What does DFS use? Why?

Discussion Points: BFS uses a Queue (FIFO) — it processes nodes level by level, and you want to finish the current level before moving to the next, so the first node you see at a level should be the first processed. DFS uses a Stack (LIFO) — either an explicit stack iteratively, or the implicit call stack via recursion.

Q5. How do you check if a binary tree is a valid BST?

Discussion Points: A naive check (left.val < root.val < right.val) is insufficient — it only checks immediate children. The correct approach: pass min and max bounds through each recursive call. For the left subtree, the upper bound is the parent's value. For the right subtree, the lower bound is the parent's value. This catches ancestors' constraints two or more levels deep.

Q6. What is the time and space complexity of the three DFS traversals?

Discussion Points: Time: O(n) for all three — each node is visited exactly once. Space: O(h) where h is the height — O(log n) for balanced trees, O(n) for skewed trees. The space comes from the recursion call stack or explicit stack depth, which is proportional to the tree height.

Q7. What are the three cases for deleting a node in a BST?

Discussion Points: Case 1: Leaf node (no children) — simply remove it. Case 2: One child — replace the node with its single child. Case 3: Two children — find the inorder successor (smallest node in the right subtree), copy its value to the node being deleted, then delete the inorder successor from the right subtree.

Q8. What is the difference between a complete binary tree, a full binary tree, and a perfect binary tree?

Discussion Points: Full: every node has 0 or exactly 2 children. Complete: all levels full except possibly the last; last level filled left to right (this is what a binary heap uses). Perfect: all internal nodes have 2 children AND all leaves are at the same level (all levels completely filled). Perfect $\subset$ Complete $\subset$ Full (in terms of strictness).

Q9. In an inorder traversal of a BST, can you find the kth smallest element? What is the complexity?

Discussion Points: Yes — inorder traversal of a BST gives nodes in sorted ascending order. Traverse inorder and count; when the counter reaches k, return that node's value. Time O(n) in the worst case, O(k + h) on average with early termination. A follow-up (LC 230 hard): maintain a counter in the recursive call to stop as soon as you find the kth element.

Q10. What is the relationship between tree height and worst-case BST performance?

Discussion Points: All BST operations (search, insert, delete) are O(h) where h = height. In a balanced BST, h = O(log n), giving O(log n) operations. In a skewed BST (worst case — e.g., inserting sorted data), h = O(n), degrading all operations to O(n) — no better than a linked list. This is why self-balancing trees (AVL, Red-Black) exist.

3.2 Coding Problems

Problem 1 — Maximum Depth of Binary Tree (LeetCode 104)

Pattern: DFS Recursion (Postorder) / BFS Level Count

Difficulty: Easy | *Est. time: 25 minutes*

Problem Statement

Given the root of a binary tree, return its maximum depth — the number of nodes along the longest path from the root node down to the farthest leaf node.

Example	Tree	Expected Output	Reasoning
1	[3,9,20,null,null,15,7]	3	3 → 20 → 15 or 3 → 20 → 7 — both paths are 3 nodes deep
2	[1,null,2]	2	Only right child exists; path is 1 → 2 = 2 nodes
3	[]	0	Empty tree has depth 0

Approach 1 — DFS Recursion (Postorder)

The depth of any node = 1 + the maximum depth of its left and right subtrees. This is a classic postorder structure: compute both children's results first, then combine at the current node. The base case is an empty node returning 0.

1. If the current node is None, return 0 (empty subtree has depth 0).
2. Recursively compute left_depth = max_depth(node.left).
3. Recursively compute right_depth = max_depth(node.right).
4. Return 1 + max(left_depth, right_depth).

```
class Solution:
    def maxDepth(self, root) -> int:
        if root is None:
            return 0                              # base case: empty node

        left_depth  = self.maxDepth(root.left)   # depth of left subtree
        right_depth = self.maxDepth(root.right)  # depth of right subtree

        return 1 + max(left_depth, right_depth)  # current node + deeper side



# One-liner version (same logic, more Pythonic)
def maxDepth_compact(root):
    if not root: return 0
    return 1 + max(maxDepth_compact(root.left), maxDepth_compact(root.right))
```

Approach 2 — BFS (Count Levels)

Process the tree level by level using a queue. Each full level processed increments the depth counter. The final counter equals the maximum depth.

```
from collections import deque

def maxDepth_bfs(root) -> int:
    if not root:
        return 0
    depth = 0
    q = deque([root])
    while q:
        depth += 1                      # entering a new level
        for _ in range(len(q)):         # process all nodes at current level
            node = q.popleft()
            if node.left:  q.append(node.left)
            if node.right: q.append(node.right)
    return depth
```

Dry Run — DFS on [3, 9, 20, null, null, 15, 7]

```
Tree:
        3
       / \
      9  20
        /  \
       15   7

Call stack trace:
maxDepth(3)
├── maxDepth(9)
│   ├── maxDepth(None) → 0
│   └── maxDepth(None) → 0
│   return 1 + max(0,0) = 1
└── maxDepth(20)
    ├── maxDepth(15)
    │   ├── maxDepth(None) → 0
    │   └── maxDepth(None) → 0
    │   return 1 + max(0,0) = 1
    └── maxDepth(7)
        ├── maxDepth(None) → 0
        └── maxDepth(None) → 0
        return 1 + max(0,0) = 1
    return 1 + max(1,1) = 2
return 1 + max(1, 2) = 3  ✓
```

Complexity Analysis

Approach	Time	Space	Notes
DFS Recursive	O(n)	O(h)	Visits every node once. Stack depth = tree height h = O(log n) balanced, O(n) skewed
BFS Iterative	O(n)	O(w)	Visits every node once. Queue holds up to one full level; max width w = O(n/2) = O(n) for last level

Follow-up Questions

- What is the minimum depth of a binary tree? (LC 111) — BFS is better here since it finds the first leaf node encountered.

- Why is DFS simpler to code here? Because the recursive structure naturally matches the recursive tree definition.

Problem 2 — Same Tree (LeetCode 100)

Pattern: DFS Simultaneous Traversal / BFS Twin Queue

Difficulty: Easy | *Est. time: 25 minutes*

Problem Statement

Given the roots of two binary trees p and q, write a function to check if they are the same. Two binary trees are considered the same if they are structurally identical and the nodes have the same values.

p tree	q tree	Output	Reason
[1,2,3]	[1,2,3]	True	Same structure, same values at every node
[1,2]	[1,null,2]	False	Same values but different structure (left vs right child)
[1,2,1]	[1,1,2]	False	Different values at same positions

Approach — Simultaneous DFS

Traverse both trees at the same time with the same traversal. At each pair of corresponding nodes, check three things:

5. If BOTH are None → they match here, return True.
6. If ONE is None and the other is not → structural mismatch, return False.
7. If both exist but their values differ → value mismatch, return False.
8. Otherwise recurse: check left subtrees AND right subtrees simultaneously.

```
class Solution:
    def isSameTree(self, p, q) -> bool:
        # Both empty → identical at this position
        if p is None and q is None:
            return True

        # One empty, one not → structural mismatch
        if p is None or q is None:
            return False

        # Both exist but values differ
        if p.val != q.val:
            return False

        # Recurse: both subtrees must match
        return (self.isSameTree(p.left,  q.left) and
                self.isSameTree(p.right, q.right))


# Compact one-liner version (same logic)
def isSameTree_compact(p, q):
    if not p and not q:  return True
    if not p or  not q:  return False
```

```
    return p.val == q.val and isSameTree_compact(p.left, q.left) and
isSameTree_compact(p.right, q.right)
```

Approach 2 — BFS with Twin Queue

Enqueue corresponding node pairs. At each step, dequeue one pair and apply the same three checks. Both trees' nodes are processed together in lockstep.

```
from collections import deque

def isSameTree_bfs(p, q) -> bool:
    queue = deque([(p, q)])

    while queue:
        node1, node2 = queue.popleft()

        if not node1 and not node2: continue          # both None → OK
        if not node1 or  not node2: return False      # structural mismatch
        if node1.val != node2.val:  return False      # value mismatch

        queue.append((node1.left,  node2.left))
        queue.append((node1.right, node2.right))

    return True
```

Dry Run — p=[1,2,3], q=[1,2,3]

Call	p node	q node	Check	Result
1	p=1	q=1	both exist, 1==1 → recurse	continue
2	p.left=2	q.left=2	both exist, 2==2 → recurse	continue
3	p.left.left=None	q.left.left=None	both None	True
4	p.left.right=None	q.left.right=None	both None	True
5	p.right=3	q.right=3	both exist, 3==3 → recurse	continue
6	p.right.left=None	q.right.left=None	both None	True
7	p.right.right=None	q.right.right=None	both None	True
Final	—	—	All checks passed	True ✓

Dry Run — p=[1,2], q=[1,null,2] (should return False)

Call	p node	q node	Check	Result
1	p=1	q=1	both exist, 1==1 → recurse	continue
2	p.left=2	q.left=None	one is None, one isn't	False ✓ — stop here

Complexity Analysis

- **Time Complexity:** O(n) — in the worst case (identical trees), every corresponding pair of nodes is visited once. n = min(nodes in p, nodes in q).

- **Space Complexity:** O(h) for DFS recursion (call stack depth = tree height). O(w) for BFS (queue width = max nodes at any level). Both are O(n) worst case for skewed trees.

Problem 3 — Invert Binary Tree (LeetCode 226)

Pattern: DFS (Postorder swap) / BFS (Level-by-level swap)

Difficulty: Easy | *Est. time: 25 minutes*

Problem Statement

Given the root of a binary tree, invert the tree (mirror it) and return its root. Every left child and its corresponding right child are swapped at every node in the tree.

```
Input:           Output:
     4                4
    / \              / \
   2   7     →      7   2
  / \ / \          / \ / \
 1  3 6  9        9  6 3  1

The left and right children of EVERY node in the tree are swapped.
```

Approach 1 — DFS Recursive (Postorder)

Invert both subtrees first, then swap the left and right pointers of the current node. Postorder is natural here: you want the children fully inverted before swapping them at the parent.

9. Base case: if node is None, return None.
10. Recursively invert the left subtree.
11. Recursively invert the right subtree.
12. Swap: node.left, node.right = node.right, node.left.
13. Return the node (now with swapped children).

```
class Solution:
    def invertTree(self, root):
        if root is None:
            return None

        # Recurse into both subtrees first (postorder)
        self.invertTree(root.left)
        self.invertTree(root.right)

        # Swap left and right children
        root.left, root.right = root.right, root.left

        return root



# Preorder version also works — swap first, then recurse
def invertTree_preorder(root):
    if not root:
        return None
    root.left, root.right = root.right, root.left   # swap first
    invertTree_preorder(root.left)
    invertTree_preorder(root.right)
```

```
    return root
```

Approach 2 — BFS (Level-by-Level Swap)

Process each node using a queue. When you dequeue a node, immediately swap its left and right children, then enqueue both (non-null) children for future processing.

```
from collections import deque

def invertTree_bfs(root):
    if not root:
        return None
    q = deque([root])
    while q:
        node = q.popleft()
        # Swap children at this node
        node.left, node.right = node.right, node.left
        # Enqueue non-null children (already swapped)
        if node.left:  q.append(node.left)
        if node.right: q.append(node.right)
    return root
```

Dry Run — DFS on [4, 2, 7, 1, 3, 6, 9]

```
Recursive call trace:
invertTree(4)
├── invertTree(2)
│   ├── invertTree(1) → swap children (None,None) → return node(1)
│   └── invertTree(3) → swap children (None,None) → return node(3)
│   swap: node(2).left=3, node(2).right=1
│   return node(2)  [now: 2 has left=3, right=1]
└── invertTree(7)
    ├── invertTree(6) → swap (None,None) → return node(6)
    └── invertTree(9) → swap (None,None) → return node(9)
    swap: node(7).left=9, node(7).right=6
    return node(7)  [now: 7 has left=9, right=6]
swap root(4): left=7 (was right), right=2 (was left)
return root(4)

Final tree:
      4
     / \
    7   2
   / \ / \
  9  6 3  1   ✓
```

Complexity Analysis

Approach	Time	Space	Notes
DFS Recursive	O(n)	O(h)	Each node visited once. Stack depth = h. O(log n) balanced, O(n) skewed
BFS Iterative	O(n)	O(w)	Each node visited once. Queue holds at most one full level

Common Mistakes to Watch For

- Forgetting the base case (None check) — without it the code will throw AttributeError on None.left.
- Only swapping at the root level and not recursing — the swap must happen at every node, not just the root.
- Inorder traversal does NOT work for inversion — you'd swap left and right, then recurse into the right (which was originally left), effectively processing the same node twice. Only preorder and postorder work correctly.

Part 4: Complete BST Implementation Task (Practical — 45 min)

Trainer: Practical Exercise

Have students build this BST class from scratch step by step. Build insert first, test it with display, then add search, then all traversals, then delete. Test after each addition.

```
from collections import deque


class TreeNode:
    def __init__(self, val):
        self.val   = val
        self.left  = None
        self.right = None


class BST:
    def __init__(self):
        self.root = None

    # ── INSERT ──────────────────────────────────────────────
    def insert(self, val):
        self.root = self._insert(self.root, val)

    def _insert(self, node, val):
        if node is None:
            return TreeNode(val)
        if val < node.val:
            node.left  = self._insert(node.left, val)
        elif val > node.val:
            node.right = self._insert(node.right, val)
        return node                        # return unchanged node if val exists

    # ── SEARCH ──────────────────────────────────────────────
    def search(self, val) -> bool:
        node = self.root
        while node:
            if val == node.val:   return True
            elif val < node.val:  node = node.left
            else:                 node = node.right
        return False

    # ── MIN / MAX ───────────────────────────────────────────
    def minimum(self):
        node = self.root
        while node and node.left:
            node = node.left
        return node.val if node else None

    def maximum(self):
        node = self.root
```

```
        while node and node.right:
            node = node.right
        return node.val if node else None

    # ── DELETE ──────────────────────────────────────────────
    def delete(self, val):
        self.root = self._delete(self.root, val)

    def _delete(self, node, val):
        if not node: return None
        if val < node.val:
            node.left  = self._delete(node.left,  val)
        elif val > node.val:
            node.right = self._delete(node.right, val)
        else:
            if not node.left and not node.right: return None      # leaf
            if not node.left:  return node.right                  # one child
            if not node.right: return node.left                   # one child
            # Two children: find inorder successor (min of right subtree)
            successor  = self._min_node(node.right)
            node.val   = successor.val
            node.right = self._delete(node.right, successor.val)
        return node

    def _min_node(self, node):
        while node.left: node = node.left
        return node

    # ── TRAVERSALS ──────────────────────────────────────────
    def inorder(self):
        result = []
        self._inorder(self.root, result)
        return result

    def _inorder(self, node, result):
        if node:
            self._inorder(node.left, result)
            result.append(node.val)
            self._inorder(node.right, result)

    def preorder(self):
        result = []
        self._preorder(self.root, result)
        return result

    def _preorder(self, node, result):
        if node:
            result.append(node.val)
            self._preorder(node.left,  result)
            self._preorder(node.right, result)

    def postorder(self):
        result = []
        self._postorder(self.root, result)
        return result
```

```
    def _postorder(self, node, result):
        if node:
            self._postorder(node.left,  result)
            self._postorder(node.right, result)
            result.append(node.val)

    def level_order(self):
        if not self.root: return []
        result, q = [], deque([self.root])
        while q:
            level = []
            for _ in range(len(q)):
                node = q.popleft()
                level.append(node.val)
                if node.left:  q.append(node.left)
                if node.right: q.append(node.right)
            result.append(level)
        return result

    # ── HEIGHT ──────────────────────────────────────────────
    def height(self):
        return self._height(self.root)

    def _height(self, node):
        if not node: return 0
        return 1 + max(self._height(node.left), self._height(node.right))

    # ── DISPLAY ─────────────────────────────────────────────
    def display(self):
        print("Inorder    :", self.inorder())
        print("Preorder   :", self.preorder())
        print("Postorder  :", self.postorder())
        print("Level Order:", self.level_order())
        print("Height     :", self.height())



# ── TEST DRIVER ─────────────────────────────────────────────
bst = BST()
for v in [5, 3, 7, 1, 4, 6, 8, 2]:
    bst.insert(v)

print("After inserting [5,3,7,1,4,6,8,2]:")
bst.display()
#          5
#         / \
#        3   7
#       / \ / \
#      1  4 6  8
#       \
#        2

print("\nSearch 4:", bst.search(4))    # True
print("Search 9:", bst.search(9))      # False
```

```
print("Min:", bst.minimum())          # 1
print("Max:", bst.maximum())          # 8

bst.delete(3)
print("\nAfter deleting 3 (two-child case):")
print("Inorder:", bst.inorder())      # [1, 2, 4, 5, 6, 7, 8]
```

Part 5: Recap, Bonus Problems & Homework

5.1 Day 3 Key Takeaways

- Tree = non-linear, hierarchical; n nodes → exactly n-1 edges; no cycles.
- Know all 19 terminologies cold — depth vs height, internal vs leaf, subtree vs path.
- BST invariant: left < node < right, holds for every node recursively — not just immediate children.
- 5 traversals: Inorder/Preorder/Postorder (DFS — Stack), Level Order / Reverse Level Order (BFS — Queue).
- Inorder of a BST = sorted sequence. This is the only traversal with that guarantee.
- BST delete has 3 cases: leaf, one child, two children (inorder successor replaces the node).
- All 3 coding problems solved in O(n) time, O(h) space — recognise the DFS recursive pattern.
- Skewed BST degrades all ops to O(n) — real systems use self-balancing trees (AVL/Red-Black).

5.2 Traversal Quick Reference Card

Traversal	Mnemonic	Order	Output on BST [4,2,6,1,3,5,7]
Preorder	Root First	Root → L → R	[4, 2, 1, 3, 6, 5, 7]
Inorder	Root Middle	L → Root → R	[1, 2, 3, 4, 5, 6, 7] ← sorted
Postorder	Root Last	L → R → Root	[1, 3, 2, 5, 7, 6, 4]
Level Order	Row by Row	BFS with Queue	[[4], [2,6], [1,3,5,7]]

5.3 Bonus / Extra Practice Problems

Problem	LC #	Difficulty	Pattern
Symmetric Tree	101	Easy	DFS simultaneous / BFS mirror check
Path Sum	112	Easy	DFS — subtract target as you descend
Diameter of Binary Tree	543	Easy	DFS — longest path through any node
Balanced Binary Tree	110	Easy	DFS — height difference check
Binary Tree Level Order Traversal	102	Medium	BFS — level-by-level with deque
Lowest Common Ancestor of BST	235	Medium	BST property — navigate without extra traversal
Lowest Common Ancestor of BT	236	Medium	DFS — find target in subtrees
Validate Binary Search Tree	98	Medium	DFS with min/max bounds
Kth Smallest Element in a BST	230	Medium	Inorder traversal with early stop
Serialize and Deserialize Binary Tree	297	Hard	Preorder DFS with None markers

5.4 Homework Assignment

Complete the following before Day 4:

14. Re-implement the complete BST class from memory (no reference) — insert, search, delete, all 4 traversals.
15. Solve LeetCode 110 (Balanced Binary Tree) — good bridge between height and tree structure.
16. Solve LeetCode 112 (Path Sum) — practice passing state down through DFS recursion.
17. Optional stretch: LeetCode 98 (Validate BST) — tests deep understanding of BST invariant with min/max bounds.